

Multi-Cloud Access Governance Dashboard

ATHAR: one permission model across AWS, Azure and GCP; measured blast radius; findings a human can act on; decisions an auditor can verify.

ABSTRACT

Government shared-services estates run across several clouds, and permissions drift: people keep access after changing roles, service accounts outlive their projects, and no console shows the whole picture. ATHAR ingests native IAM exports from three providers plus an HR feed, links the same person across three identifier schemes, and normalises 104,593 provider grants into one canonical model. Pure detection rules flag wildcard privilege, dormant access, departed staff, cross-cloud superusers and toxic combinations, each citing the rows behind it. Risk is a measured blast radius over an access graph spanning all three clouds, shown as arithmetic. Two things are unusual: a fenced agent layer drafts least-privilege remediation but can never create a finding, change a score or act outside the permitted set, and every scan and decision is committed to a Merkle-anchored ledger, so a report verifies against the chain without trusting the tool. It runs from the README in under three minutes, runs cleanly a second time, and was evaluated on a seed its scoring constants never saw.

TEAM

ROLE	NAME	EMAIL
Academic advisor	Dr Ashish Gupta	ashish@dubai.bits-pilani.ac.in
Captain	Raghubir Krishna Shankar Prasad	f20240125@dubai.bits-pilani.ac.in
Member	Sachin Putlur Premkumar	f20230208@dubai.bits-pilani.ac.in
Member	Nishal Ahmed Poovatham Kandiyil	f20230209@dubai.bits-pilani.ac.in
Member	Sai Uday Kodithyala	f20230254@dubai.bits-pilani.ac.in
Member	Mohammed Ruknuddin	f20230902@dubai.bits-pilani.ac.in
Institution	BITS Pilani, Dubai Campus	

WHERE TO FIND IT

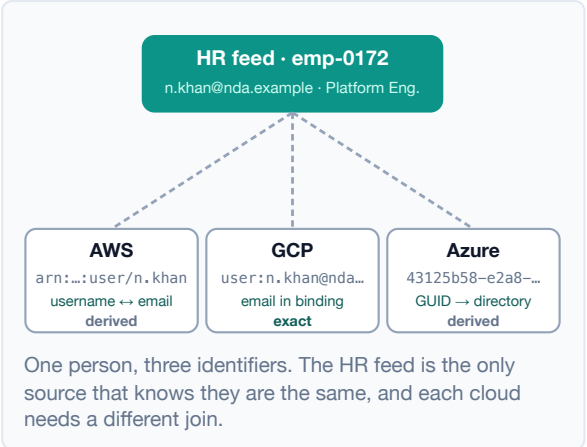
GITHUB	https://github.com/raghubirPrasad/athar source, README, tests and this report
GOOGLE DRIVE	https://drive.google.com/drive/folders/1TJAL03HBtdECItQyJV-XP0uuCIGEbcaV holds a zip of the code and the demo video
RUN IT	cp .env.example .env && make demo → http://localhost:8080

CONTENTS	1 Project objective · 2 Proposed solution · 3 Solution validation · 4 Results and conclusions
DATA	Entirely synthetic. The customer, Nahar Digital Authority, is fictional; addresses use the reserved nda.example domain; no real account, entity or person is referenced.

1 Project objective

1.1 THE PROBLEM THE BRIEF DESCRIBES

A government body running AWS, Azure and GCP has three separate answers to “who can do what” and no way to add them up. Access is granted at every hire, role change and incident and revoked almost never: on the simulated estate, one department was granted 500 permissions in a year and had 27 removed when people left. A spreadsheet review looks at people one at a time and cannot see a whole team holding administrative power in all three clouds. The objective is one dashboard that ingests permission data from three simulated providers into one model, flags over-privileged users, orphaned accounts and risky combinations, explains why, and produces a report someone can act on.



1.2 WHO BUYS IT, WHO DEPLOYS IT, WHAT IT REPLACES

BUYER	The head of cyber security of a federal or emirate-level shared-services authority running citizen-facing services on more than one cloud. Internal audit is the second signature.
DEPLOYS	The cloud security team, on premises or in an approved region, fed by scheduled IAM exports and an HR feed. Nothing is installed on the provider side and no provider credential is held.
USES	A security engineer at the evidence altitude, a governance officer at the explanation altitude, a director or auditor at the headline altitude with the half-life table.
REPLACES	Quarterly spreadsheet access reviews; three consoles read side by side; per-identity CIEM licences that stop at “who is over-privileged today” and keep no attested record of what was decided.

1.3 THE GRADED MINIMUM, MAPPED

BRIEF REQUIRES	DELIVERED AS
Synthetic, realistic IAM data for three providers, different native formats	425 export files a month in the providers’ own shapes: AWS authorization details and credential report, Azure role assignments, role definitions and Entra directory, one GCP IAM policy per project, service-account keys, plus an HR feed. A judge can open the raw JSON.
A normaliser into one common permission model	One canonical model of identities, principals, grants, activity, credentials and resources; each provider grant expands into a row per verb and service category, and every row keeps its source file and a JSON pointer to the bytes it came from.
Detection: admin/wildcard, unused access, departed staff, same power across three clouds, toxic combinations	All five, plus unmapped permissions, stale credentials, privileged humans without MFA, data-residency drift and unowned principals. A rule that cannot cite concrete rows may not emit a finding.
Dashboard: identity list with risk scores, drill-down into why, filters by cloud and department	Ranked table of 507 identities; drill-down with score line items, escalation paths, causal history and raw evidence; filters by cloud, department, rule, severity and status.
An exportable findings report	CSV with a recommended action per row and a PDF for the board, both carrying the Merkle root and transaction, both verifiable against the chain without the dashboard.

1.4 WHY THIS APPROACH NOW, RATHER THAN THE ONE OF THREE YEARS AGO

Multi-cloud is the default, not the exception. A per-provider tool was once a defensible answer. Today the join across providers is the problem itself: the finding that matters most on this estate, one team holding the same power in all three clouds, does not exist inside any single provider’s view.

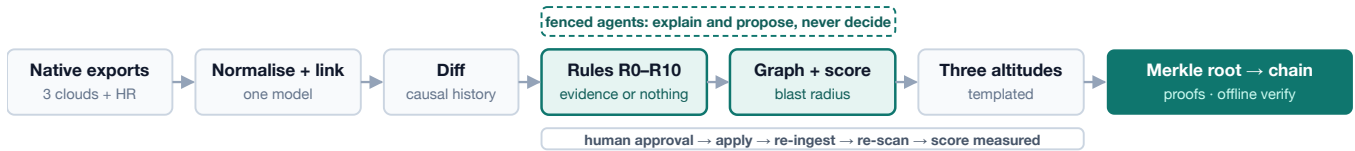
Language models can now draft a remediation, so the design question has moved. It is no longer whether an agent may propose but what it must never decide. The rule engine owns every finding, severity, score and permitted action; the model explains and proposes inside those limits, and its proposals are validated before a human sees them.

Attested audit trails are now expected, and cheap. Regulators ask whether the record has been edited since it was produced; anchoring a Merkle root per scan costs a few hundred bytes and is verifiable with standard tooling.

2 Proposed solution

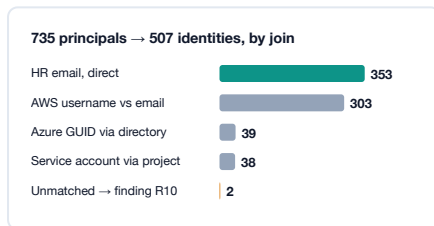
2.1 ARCHITECTURE

A pipeline of pure stages with side effects at the edges. Exports are normalised and upserted on natural keys, so re-ingesting a month changes nothing. Each scan diffs the month against the last to write the causal history, runs the rules over a read-only estate view, builds an access graph, scores every identity, renders three altitudes of narrative from templates, hashes each finding into a Merkle tree and commits the root. The API serves what the pipeline computed.



2.2 ONE IDENTITY ACROSS THREE SCHEMES

The HR feed is the anchor. Four joins in descending confidence; two principals matched nothing and became findings rather than being attached to somebody. An identifier matching more than one HR row is dropped rather than taken first, because a merged identity has a footprint neither real one has.



2.3 DETECTION AND SCORING

Rules consult the governance register for exceptions and never a cloud-side tag. Blast radius is the share of resources reachable within four hops of a graph spanning all three clouds, high-sensitivity weighted three times. Every term is a stored line item.

```
reach = min(1, blast_radius / 0.25)
exploitability = 1 + departed .5 + dormant .3 + no-MFA .3
                + external .2 + old key .2 + cross-cloud .2
controls = 1 - register credits (.35 / .20 / .15)
score = max(100 · reach · exploit · controls, rule floor) ≤ 100
```

Two real scores, as the drill-down shows them

A departed engineer, still holding access

reach 1.0 × exploitability 1.8 × controls 1.0 = 180 floor 75 (R3)

→ 100 Critical

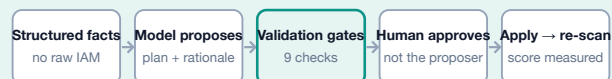
A decoy with break-glass on the register

reach 1.0 × exploitability 1.0 × controls 0.65 = 65 no rule fired

→ 65 High

2.4 WHAT IS DIFFERENT

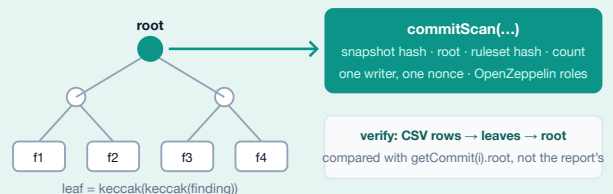
The fence: the engine decides, the model explains



rejected → deterministic diff instead
action outside the rule's set · grant not shown ·
grant unaccounted for · kept but unused

Agentic AI, fenced. **HIGHLIGHT** Three agents investigate, plan and summarise from structured facts, never raw IAM. A plan naming an action outside the rule's set, touching a grant it was not shown, leaving one unaccounted for, or keeping one that is unused is rejected before a human sees it. Reduction is recomputed by rebuilding the graph. Output passes an injection guard; results are cached with a template fallback, so the demo needs no network.

The ledger: hashes on chain, never data

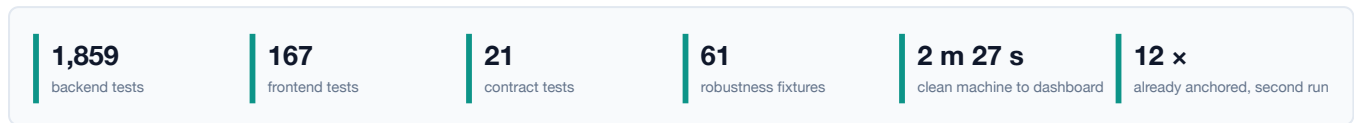


Blockchain governance ledger. **HIGHLIGHT** Each scan commits four hashes and a count; no IAM data goes on chain. Every finding carries its own proof, every approval is recorded against that proof under role-separated access control, and a CSV verifies against the root read from the chain rather than the one printed in the report. The interface states what this protects against and what it does not.

Permission Half-Life turns dozens of findings into one organisational number per department (granted vs revoked on departure, and how long a revoked grant survived). **A process-simulated estate** means over-privilege is emergent rather than staged, reproducible byte for byte from a seed, with each finding carrying the month and event that created it.

3 Solution validation

Every claim here was run, not asserted. Where a claim could not be tested, it is not made.

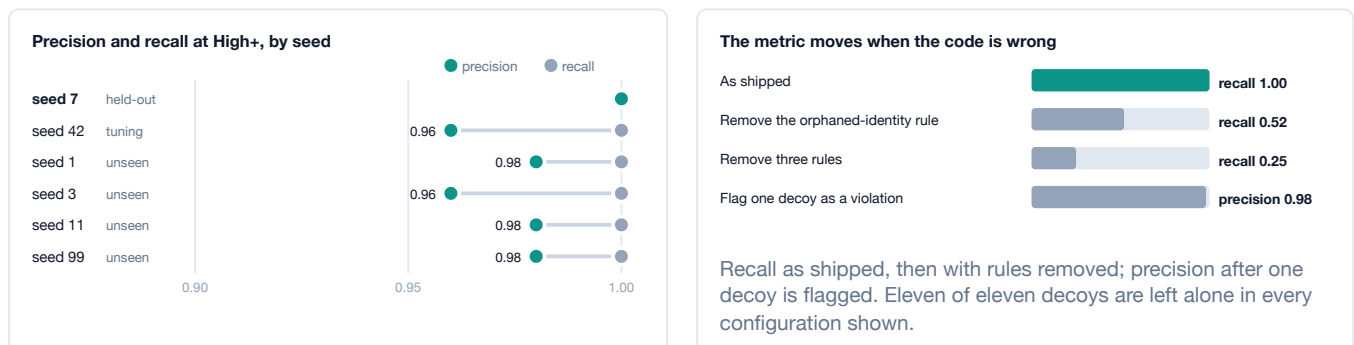


3.1 IT RUNS END TO END FROM THE README, AND AGAIN

From a zip of the committed tree on a machine with only Docker and make: copy the template, run make demo, open the dashboard. The stack starts Postgres, a local chain, the API and the web app, generates the estate, ingests and scans twelve months anchoring each. A second run prints twelve lines ending *already anchored*: nothing duplicated, no contract redeployed. The month-12 root is identical on the host and in Docker.

3.2 IT SURVIVES UNEXPECTED INPUT, AND THE SCENARIO COMPLETES

Uploads are schema-validated: a malformed file, a non-UTF-8 file, a file for the wrong provider and a policy with an unknown action all return a structured problem response with a stable code and no stack trace; an unknown role becomes an unmapped-permission finding, not an exception. The brief's scenario runs on the 500-identity estate: sorted by risk, the top three are one team at score 100 and blast radius 69.9 percent, all on the cross-cloud superuser rule, with the raw provider JSON behind each. The loop then closes: a plan for a departed engineer, approved by a second account, applied, re-ingested and re-scanned, moved that identity from 100 to 0 with both decisions anchored. Editing one stored severity fails verification with both roots printed and turns the badge red on its own; re-running the month restores it.



3.3 CRYPTOGRAPHY, BY THE BOOK

Leaves are `keccak256(keccak256(canonical_json))` and nodes hash sorted pairs, giving domain separation between leaves and nodes and matching OpenZeppelin's MerkleProof; one finding's seven-node proof was checked against the contract's own verify function and returned true. The contract uses OpenZeppelin AccessControl with separate roles for committing scans and recording decisions. **One writer owns the transaction nonce and nothing else signs**; a job parked by a node outage is re-checked against the chain before re-send, so one root is never committed twice. The development key is the local chain's publicly known account, labelled as such, and no secret is in the repository.

3.4 STATISTICS, WITHOUT LEAKAGE

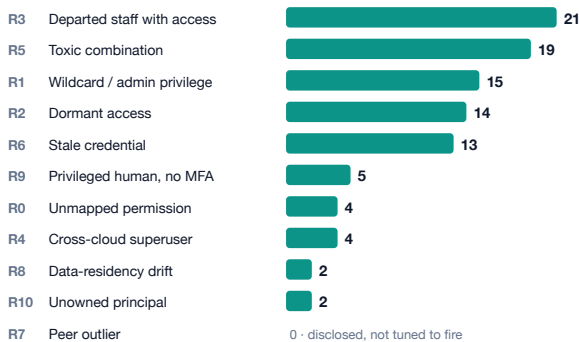
Ground truth is written by the generator from its own bookkeeping as a second statement of each rule's predicate; the evaluation asks whether the engine recovers it from the exported files alone, at High severity and above, with a per-rule confusion table beneath. **Constants are tuned only on seed 42 and reported on seed 7, which they never saw**, and the result file records which seed it came from, so the tuning seed can never be labelled held-out. The decoys, legitimate only through a register entry, test judgement rather than recall.

3.5 IT WAS ATTACKED BEFORE IT WAS SUBMITTED

The fence was tested against a real local model: it proposed a plan touching grants it had not been shown, the engine rejected it with four named violations and used the deterministic diff, and findings, scores and root were byte-identical afterwards. Two rounds of adversarial review then found, and we fixed: explicit-deny rows never reaching the model, the planner able to propose deleting a guardrail, a leaked database lock that would have stopped continuous governance after one run, a parked ledger job able to commit a root twice, and an open redirect at login.

4 Results and conclusions

Open findings by rule, month 12



One approved remediation, measured before and after



99 open findings by severity



Permissions granted vs revoked on departure, by department



The organisational finding. A department revoking under a tenth of what it grants has no half-life at all. One department in nine revokes often enough to measure, and still takes five months.

382 + 125

people and service accounts
at month 12

104,593

canonical grant rows from 425
export files

99 → 96

open findings after one
approved remediation

144

explicit deny rows retained
and honoured

The top three by risk are one team holding the same power in AWS, Azure and GCP. The twenty-one orphaned accounts are one offboarding process. Neither is visible to any single console, and neither is a list of people.

4.1 WHAT IS NOT CLAIMED

- The held-out evaluation is a round-trip check on the pipeline: truth and rules share their definitions by construction, so a perfect score is expected of working code, not suspicious. A real estate has no ground truth, which is why real tools cannot publish one.
- Applying a remediation changes the simulated estate, deliberately; no provider SDK is in the dependencies. The production step replaces one function with a pull request against infrastructure-as-code. The plan, the fence, separation of duties and the anchored decision all stay.
- The ledger defends against post-hoc editing of findings and decisions and disputes over who approved what. It does not defend against a compromised host holding the writer key; production needs a hardware-backed key and per-approver wallets.
- One rule, peer outlier, fires on nothing because the simulated estate saturates every department's breadth. It is measured and disclosed rather than tuned to fire.

4.2 CONCLUSIONS

The graded minimum is met and exercised end to end, and the prototype survives the two things a judge will do first: run it twice and feed it something broken. The differentiators are not complexity. A half-life number a director can act on; an estate that rotted on its own rather than being staged; an agent that can propose but never decide; and a record an auditor can verify without trusting the tool. The cryptography follows the standard construction with a single nonce owner, the statistics keep a clean split between tuning and reporting with provenance recorded in the result, and every number in this report was read back from a clean run and reproduces from the same seed. Deploying it takes one function and a key ceremony, not a redesign.